Software Engineering (IS51) – Units 1 & 2: Important Topics and PPT Answers

Based only on your PPTs (Unit_1_1, Unit_1_2, Unit_1_3, Unit_2_1) and the four papers you uploaded: Feb 2024, Feb/Mar 2025 (backlog), Feb/Mar 2025 (IS53 OOO), Jan 2026.

Slide key: **U1.1** = Unit_1_1 (Ch 1 Introduction), **U1.2** = Unit_1_2 (Ch 2 Software Processes), **U1.3** = Unit_1_3 (Ch 3 Agile), **U2.1** = Unit_2_1 (Ch 4 Requirements Engineering). Unit_2_2 (Ch 5 System Modeling) is only needed for use-case ideas, which are Unit 3 style questions.

Part A – What to study, with slide numbers

Tier 1 – Must do (asked in 3 or 4 of the 4 papers)

#	Topic	Asked	Slides
1	RUP: four phases with diagram (dynamic perspective)	4 of 4	U1.2: 53–56 (diagram on 54); 57–60 for workflows and good practice
2	Plan-driven vs agile (also hybrid, use case)	4 of 4	U1.3: 4–5, 10–14; U1.2: 5
3	Functional vs non-functional requirements, types of requirements, types of NFR	4 of 4	U2.1: 6, 9–10, 14, 17–22 (NFR types diagram on 15)
4	Requirements validation and four checks	3 of 4	U2.1: 70–74
5	Extreme programming: need and cycle	3 of 4	U1.3: 15–19 (release cycle diagram on 17), 20, 23–25
6	Requirements engineering process and its activities	3 of 4	U2.1: 46–47 (spiral diagram on 47), 50–52
7	Requirements document structure and ways of writing system requirements	3 of 4	U2.1: 27, 31–34
8	Waterfall model with advantages and limitations	2 of 4	U1.2: 6–10 (diagram on 7)

Tier 2 – Do next (asked once or twice)

#	Topic	Slides
9	Requirements elicitation, discovery and analysis process	U2.1: 48–53, 56–61
10	Ethnography and prototyping for requirements analysis	U2.1: 66–69; prototyping on U1.2: 39–43
11	Ethical responsibilities of software engineers	U1.1: 23–30
12	Spiral model and its advantages over waterfall	U1.2: 49–52
13	Incremental development, fundamental activities	U1.2: 11–18, 22; delivery on 44–48
14	Problems with natural language, structured specifications	U2.1: 36–45
15	Change management process	U2.1: 75–81 (diagram on 81)
16	Key principles of professional software development	U1.1: 10–11, 18
17	Insulin pump activity model	U1.1: 32–35 (activity diagram on 34)

Tier 3 – Skip unless time is left

- SRS characteristics of a good SRS: not in your PPTs.
- NFR lists for real-time banking, traffic signal, e-commerce: not in your PPTs, but follow the NFR format on U2.1 slides 17–21 (product, organisational, external, with measures).
- Software engineering concepts for a database information system: use U1.1 slides 3, 10–12 and the MHC-PMS slides 36–41.

Part B – Answers from the PPTs

Wording follows the slides. Figures marked *(draw)* are images in the slides, so redraw them from the slide number given.

1. RUP: four phases (U1.2 slides 53–56)

- The Rational Unified Process is a modern generic process derived from the work on the UML and associated process. It brings together aspects of the 3 generic process models discussed previously.

- Normally described from 3 perspectives: a dynamic perspective that shows phases over time; a static perspective that shows process activities; a practice perspective that suggests good practice.

(draw: 'Phases in the Rational Unified Process', slide 54: Inception → Elaboration → Construction → Transition, with the loop back)

Phases

- **Inception:** Establish the business case for the system.
- **Elaboration:** Develop an understanding of the problem domain and the system architecture.
- **Construction:** System design, programming and testing.
- **Transition:** Deploy the system in its operating environment.

Iteration

- **In-phase iteration:** Each phase is iterative with results developed incrementally.
- **Cross-phase iteration:** As shown by the loop in the RUP model, the whole set of phases may be enacted incrementally.

Static workflows (slides 57–58): Business modelling, Requirements, Analysis and design, Implementation, Testing, Deployment, Configuration and change management, Project management, Environment.

RUP good practice (slides 59–60): Develop software iteratively; Manage requirements; Use component-based architectures; Visually model software; Verify software quality; Control changes to software.

RUP vs agile (for the advantage/disadvantage question): use slides 59–60 for RUP strengths (iterative, documented requirements, visual UML models, quality verification, change control) and U1.3 slides 4–8 for agile (reduce overheads by limiting documentation, respond quickly to change; problems: customer interest, team suitability, simplicity needs extra work, contracts).

2. Plan-driven vs agile (U1.3 slides 4–5, 10–14; U1.2 slide 5)

- **Plan-driven development:** A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance. Not necessarily waterfall model – plan-driven, incremental development is possible. Iteration occurs within activities. Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.

- **Agile development:** Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process. In agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- Agile methods focus on the code rather than the design, are based on an iterative approach to software development, and are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- In practice, most practical processes include elements of both plan-driven and agile approaches. There are no right or wrong software processes.

Agile manifesto (slide 5): Individuals and interactions over processes and tools; Working software over comprehensive documentation; Customer collaboration over contract negotiation; Responding to change over following a plan.

Agile principles (slide 6): Customer involvement, Incremental delivery, People not process, Embrace change, Maintain simplicity.

Limitations of agile (slides 7–9): It can be difficult to keep the interest of customers who are involved in the process. Team members may be unsuited to the intense involvement that characterises agile methods. Prioritising changes can be difficult where there are multiple stakeholders. Maintaining simplicity requires extra work. Contracts may be a problem. Because of their focus on small, tightly-integrated teams, there are problems in scaling agile methods to large systems.

Choosing the balance / hybrid for a real-time system (slides 12–14):

- Is it important to have a very detailed specification and design before moving to implementation? If so, you probably need to use a plan-driven approach.
- Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using agile methods.
- How large is the system? Agile methods are most effective when the system can be developed with a small co-located team.
- What type of system? Plan-driven approaches may be required for systems that require a lot of analysis before implementation (e.g. real-time system with complex timing requirements).
- Other factors: expected system lifetime, available technologies, how the team is organized, cultural issues, skill of designers and programmers, and external regulation.

3. Functional vs non-functional requirements (U2.1 slides 6, 9–22)

Types of requirement (slide 6)

- **User requirements:** Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- **System requirements:** A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

Slide 9

- **Functional requirements:** Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. May state what the system should not do.
- **Non-functional requirements:** Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc. Often apply to the system as a whole rather than individual features or services.
- **Domain requirements:** Constraints on the system from the domain of operation.

Functional (slides 10–11): Describe functionality or system services. Depend on the type of software, expected users and the type of system where the software is used. Example (MHC-PMS): A user shall be able to search the appointments lists for all clinics.

Non-functional (slides 14–21): These define system properties and constraints e.g. reliability, response time and storage requirements. Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless. They may affect the overall architecture of a system rather than the individual components.

(draw: 'Types of nonfunctional requirement', slide 15)

Classification (slide 17)

- **Product requirements:** the delivered product must behave in a particular way e.g. execution speed, reliability.
- **Organisational requirements:** a consequence of organisational policies and procedures e.g. process standards used, implementation requirements.
- **External requirements:** factors external to the system and its development process e.g. interoperability requirements, legislative requirements.

Examples in the MHC-PMS (slide 18): Product: The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Organizational: Users of the MHC-PMS system shall authenticate themselves using their health authority identity card. External: The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Goals vs verifiable requirements (slides 19–20): Goal: a general intention of the user such as ease of use. Verifiable non-functional requirement: a statement using some measure that can be objectively tested.

Metrics (slide 21): Speed (transactions/second, response time), Size (Mbytes), Ease of use (training time), Reliability (mean time to failure, availability), Robustness (time to restart after failure), Portability (number of target systems).

Domain requirements (slides 22–24): The system's operational domain imposes requirements on the system. Problems: Understandability, Implicitness.

4. Requirements validation and four checks (U2.1 slides 70–74)

- Requirements validation is concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important. Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking (slide 71)

1. **Validity.** Does the system provide the functions which best support the customer's needs?
2. **Consistency.** Are there any requirements conflicts?
3. **Completeness.** Are all functions required by the customer included?
4. **Realism.** Can the requirements be implemented given available budget and technology?
5. **Verifiability.** Can the requirements be checked?

Techniques (slide 72): Requirements reviews (systematic manual analysis of the requirements); Prototyping (using an executable model of the system to check requirements); Test-case generation (developing tests for requirements to check testability).

Reviews (slides 73–74): Regular reviews should be held while the requirements definition is being formulated. Both client and contractor staff should be involved. Reviews may be formal or informal. Review checks: Verifiability, Comprehensibility, Traceability, Adaptability.

5. Extreme programming: need and cycle (U1.3 slides 15–20, 23–25)

Need / what it is

- Perhaps the best-known and most widely used agile method. Extreme Programming (XP) takes an 'extreme' approach to iterative development.

- Rapid development and delivery is now often the most important requirement; businesses operate in a fast-changing requirement and it is practically impossible to produce a set of stable software requirements (slide 3).
- XP and change (slide 23): XP maintains that anticipating changes is not worthwhile as changes cannot be reliably anticipated. Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

Cycle (slides 15, 17)

- New versions may be built several times per day.
- Increments are delivered to customers every 2 weeks.
- All tests must be run for every build and the build is only accepted if tests run successfully.

(draw: 'The extreme programming release cycle', slide 17)

XP and agile principles (slide 16): Incremental development is supported through small, frequent system releases. Customer involvement means full-time customer engagement with the team. People not process through pair programming, collective ownership and a process that avoids long working hours. Change supported through regular system releases. Maintaining simplicity through constant refactoring of code.

XP practices (slides 18–19): Incremental planning, Small releases, Simple design, Test-first development, Refactoring, Pair programming, Collective ownership, Continuous integration, Sustainable pace, On-site customer.

Requirements (slide 20): User requirements are expressed as scenarios or user stories. These are written on cards and the development team break them down into implementation tasks. The customer chooses the stories for inclusion in the next release.

6. Requirements engineering process and activities (U2.1 slides 3, 46–47, 50–52)

- Requirements engineering is the process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.
- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements. However, there are a number of generic activities common to all processes: Requirements elicitation; Requirements analysis; Requirements validation; Requirements management.
- In practice, RE is an iterative activity in which these processes are interleaved.

(draw: 'A spiral view of the requirements engineering process', slide 47)

Software specification (U1.2 slide 23): Feasibility study (Is it technically and financially feasible to build the system?); Requirements elicitation and analysis (What do the system stakeholders require or expect from the system?); Requirements specification (Defining the requirements in detail); Requirements validation (Checking the validity of the requirements).

Requirements management (slides 75–79): the process of managing changing requirements during the requirements engineering process and system development.

7. Requirements document and ways of writing requirements (U2.1 slides 27, 30–34)

- The software requirements document is the official statement of what is required of the system developers. Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set out WHAT the system should do rather than HOW it should do it.

Structure (slides 31–32): Preface; Introduction; Glossary; User requirements definition; System architecture; System requirements specification; System models; System evolution; Appendices; Index. (Descriptions are on the slides.)

Ways of writing a system requirements specification (slide 34)

- **Natural language:** numbered sentences; each sentence should express one requirement.
- **Structured natural language:** natural language on a standard form or template; each field provides information about an aspect of the requirement.
- **Design description languages:** a language like a programming language, but with more abstract features; now rarely used although useful for interface specifications.
- **Graphical notations:** graphical models supplemented by text annotations; UML use case and sequence diagrams are commonly used.
- **Mathematical specifications:** based on finite-state machines or sets; unambiguous, but most customers don't understand a formal specification.

8. Waterfall model (U1.2 slides 6–10)

- The waterfall model is an example of a plan-driven process. It has separate and distinct phases of specification and development.

(draw: 'The waterfall model', slide 7)

Phases: Requirements analysis and definition → System and software design → Implementation and unit testing → Integration and system testing → Operation and maintenance.

Limitations

- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.
- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
- Few business systems have stable requirements.

Advantages / where suited (slide 10): This model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process. It is mostly used for large systems engineering projects where a system is developed at several sites; the plan-driven nature of the waterfall model helps coordinate the work.

9. Requirements elicitation, discovery and analysis (U2.1 slides 48–61)

- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints. May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders.

Stages: Requirements discovery; Requirements classification and organization; Requirements prioritization and negotiation; Requirements specification.

(draw: 'The requirements elicitation and analysis process', slide 51)

Process activities (slide 52)

- **Requirements discovery:** Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- **Requirements classification and organisation:** Groups related requirements and organises them into coherent clusters.
- **Prioritisation and negotiation:** Prioritising requirements and resolving requirements conflicts.
- **Requirements specification:** Requirements are documented and input into the next round of the spiral.

Problems (slides 49, 53): Stakeholders don't know what they really want. Stakeholders express requirements in their own terms. Different stakeholders may have conflicting requirements. Organisational and political factors may influence the system

requirements. The requirements change during the analysis process.

Requirements discovery (slides 56–61): The process of gathering information about the required and existing systems and distilling the user and system requirements from this information. Example of your choice: stakeholders in the MHC-PMS (slides 57–58) – Patients, Doctors, Nurses, Medical receptionists, IT staff, a medical ethics manager, Health care managers, Medical records staff.

Techniques

- **Interviewing (59–60):** Closed interviews based on pre-determined list of questions; open interviews where various issues are explored. Good for getting an overall understanding of what stakeholders do; not good for understanding domain requirements.
- **Scenarios (61):** real-life examples of how a system can be used. Include the starting situation, normal flow of events, what can go wrong, other concurrent activities, and the state when the scenario finishes.
- **Use cases (64):** a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- **Ethnography (66–69):** see next answer.

10. Ethnography and prototyping (U2.1 slides 66–69; U1.2 slides 39–43)

Ethnography

- A social scientist spends a considerable time observing and analysing how people actually work. People do not have to explain or articulate their work. Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.
- Requirements derived from the way that people actually work rather than the way in which process definitions suggest that they ought to work, and from cooperation and awareness of other people's activities.
- Ethnography is effective for understanding existing processes but cannot identify new features that should be added to a system.

Focused ethnography (slide 68): Developed in a project studying the air traffic control process. Combines ethnography with prototyping. Prototype development results in unanswered questions which focus the ethnographic analysis. The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

(draw: 'Ethnography and prototyping for requirements analysis', slide 69)

Prototyping (U1.2 slides 39–43): A prototype is an initial version of a system used to demonstrate concepts and try out design options. Used in requirements elicitation and validation, in design to explore options and develop a UI design, and in testing for back-to-back tests. Benefits: improved system usability, a closer match to users' real needs, improved design quality, improved maintainability, reduced development effort.

11. Ethical responsibilities (U1.1 slides 23–30)

- Software engineering involves wider responsibilities than simply the application of technical skills. Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- **Confidentiality:** Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- **Competence:** Engineers should not misrepresent their level of competence. They should not knowingly accept work which is out of their competence.
- **Intellectual property rights:** Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc.
- **Computer misuse:** Software engineers should not use their technical skills to misuse other people's computers.

ACM/IEEE Code of Ethics – Eight Principles (slide 29): 1. PUBLIC – act consistently with the public interest. 2. CLIENT AND EMPLOYER – act in the best interests of their client and employer consistent with the public interest. 3. PRODUCT – products and related modifications meet the highest professional standards possible. 4. JUDGMENT – maintain integrity and independence in professional judgment. 5. MANAGEMENT – managers and leaders subscribe to and promote an ethical approach to the management of software development and maintenance. 6. PROFESSION – advance the integrity and reputation of the profession consistent with the public interest. 7. COLLEAGUES – be fair to and supportive of colleagues. 8. SELF – participate in lifelong learning regarding the practice of their profession and promote an ethical approach to the practice of the profession.

12. Spiral model (U1.2 slides 49–52)

- Process is represented as a spiral rather than as a sequence of activities with backtracking. Each loop in the spiral represents a phase in the process.
- No fixed phases such as specification or design – loops in the spiral are chosen depending on what is required. Risks are explicitly assessed and resolved throughout the process.

(draw: slide 50)

Sectors: Objective setting; Risk assessment and reduction; Development and validation; Planning.

Over waterfall: The spiral model has been very influential in helping people think about iteration in software processes and introducing the risk-driven approach to development, whereas waterfall has inflexible partitioning into distinct stages (slide 10). **Disadvantage:** In practice, the model is rarely used as published for practical software development (slide 52).

13. Incremental development (U1.2 slides 11–18, 22, 44–48)

- The Incremental Model is a software development approach where the product is designed, implemented, and tested incrementally (in small, manageable chunks) until the product is finished. Each increment adds some functional capability to the system until the full system is implemented.
- **Benefits:** the cost of accommodating changing customer requirements is reduced; much less analysis and documentation has to be redone than with waterfall; easier to get customer feedback; more rapid delivery and deployment of useful software.
- **Problems:** the process is not visible; system structure tends to degrade as new increments are added.
- **Fundamental activities (slide 22):** specification, development, validation and evolution, which are inter-leaved in incremental development (organized in sequence in waterfall).
- Example: the ShopFast e-commerce website, slides 15–18 (Increment 1 core product and basic purchase flow; Increment 2 enhanced user experience and management; Increment 3 advanced features and scalability).

14. Natural language problems and structured specifications (U2.1 slides 36–45)

- **Problems with natural language:** Lack of clarity – precision is difficult without making the document difficult to read. Requirements confusion – functional and non-functional requirements tend to be mixed-up. Requirements amalgamation – several different requirements may be expressed together.
- **Structured specifications:** An approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way. Works well for some types of requirements e.g. embedded control system, but is sometimes too rigid for business systems.
- **Form-based specification (slide 41):** definition of the function or entity; description of inputs and where they come from; description of outputs and where they go to; information needed for the computation; description of the action to be taken; pre and post conditions; the side effects.

- **Tabular specification (slides 44–45):** used to supplement natural language, particularly useful when you have to define a number of possible alternative courses of action.

15. Change management (U2.1 slides 75–81)

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development. New requirements emerge as a system is being developed and after it has gone into use.
- **Principal stages (slide 80):** Problem analysis and change specification – the change proposal is analyzed to check that it is valid; Change analysis and costing – the effect of the proposed change is assessed using traceability information, then a decision is made whether or not to proceed; Change implementation – the requirements document and, where necessary, the system design and implementation, are modified.

(draw: 'Requirements change management', slide 81)

16. Key principles of professional software development (U1.1 slides 10–11, 18)

Some fundamental principles apply to all types of software system, irrespective of the development techniques used:

- Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
- Dependability and performance are important for all types of system.
- Understanding and managing the software specification and requirements (what the software should do) are important.
- Where appropriate, you should reuse software that has already been developed rather than write new software.

Essential attributes (slide 9): Maintainability, Dependability and security, Efficiency, Acceptability.

17. Insulin pump activity model (U1.1 slides 32–35)

- Collects data from a blood sugar sensor and calculates the amount of insulin required to be injected. Calculation based on the rate of change of blood sugar levels. Sends signals to a micro-pump to deliver the correct dose of insulin.
- Safety-critical system as low blood sugars can lead to brain malfunctioning, coma and death; high-blood sugar levels have long-term consequences such as eye and kidney damage.

(draw: 'Activity model of the insulin pump', slide 34)

Essential high-level requirements (slide 35): The system shall be available to deliver insulin when required. The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.

Suggested order for revision

1. RUP (answer 1) and Plan-driven vs agile (answer 2).
2. Functional vs non-functional (answer 3) and Requirements validation (answer 4).
3. XP (answer 5), RE process (answer 6), Requirements document (answer 7), Waterfall (answer 8).
4. Then Tier 2 topics, starting with elicitation and ethnography.

Part C – CIE-1 (Internal Assessment I) answers

CIE-1 paper: 28/10/2025, 30 marks, answer any TWO full questions (15 × 2), portions L1–L19. Q1, Q2 and Q3 are each 15 marks, so prepare any two fully; Q1 and Q2 are the safest because the slides cover them most directly. Full answers below, in the same slide wording as the PPTs.

Q1.a (8 marks) Define Software Engineering. How is it more than "just programming"? Consider a client has a vague idea of their requirements and expects them to change frequently. Which process model would you recommend and why?

Definition: Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.

- **Engineering discipline:** Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
- **All aspects of software production:** Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

How it is more than "just programming":

- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project.
- For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- Software engineering covers the four fundamental activities: software specification, software development, software validation and software evolution.
- Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.

Recommended process model: Incremental development (supported by prototyping and agile methods)

Why not waterfall: Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements. The waterfall model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process. Few business systems have stable requirements.

Why incremental development:

- Incremental Model is a software development approach where the product is designed, implemented, and tested incrementally (in small, manageable chunks) until the product is finished. Each increment adds some functional capability to the system until the full system is implemented.
- The cost of accommodating changing customer requirements is reduced.
- The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented.
- More rapid delivery and deployment of useful software to the customer is possible.
- Change tolerance: the process is designed so that changes can be accommodated at relatively low cost.
- For a **vague idea of requirements**, a prototype is an initial version of a system used to demonstrate concepts and try out design options. It helps with requirements elicitation and validation, and gives a closer match to users′ real needs.
- Agile methods are intended to deliver working software quickly and evolve this quickly to meet changing requirements. Their principle "Embrace change" says: Expect the system requirements to change and so design the system to accommodate these changes.

Q1.b (7 marks) Differentiate between Functional and Non-Functional Requirements (NFRs). Provide two examples of each for an e-commerce website.

Functional requirements	Non-functional requirements
Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.	Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
Describe functionality or system services.	Define system properties and constraints e.g. reliability, response time and storage requirements.
May state what the system should not do.	Often apply to the system as a whole rather than individual features or services.
Depend on the type of software, expected users and the type of system where the software is used.	May be more critical than functional requirements. If these are not met, the system may be useless.
Functional user requirements may be high-level statements of what the system should do. Functional system requirements should describe the system services in detail.	Classified as Product requirements, Organisational requirements and External requirements.

Examples for an e-commerce website

Functional:

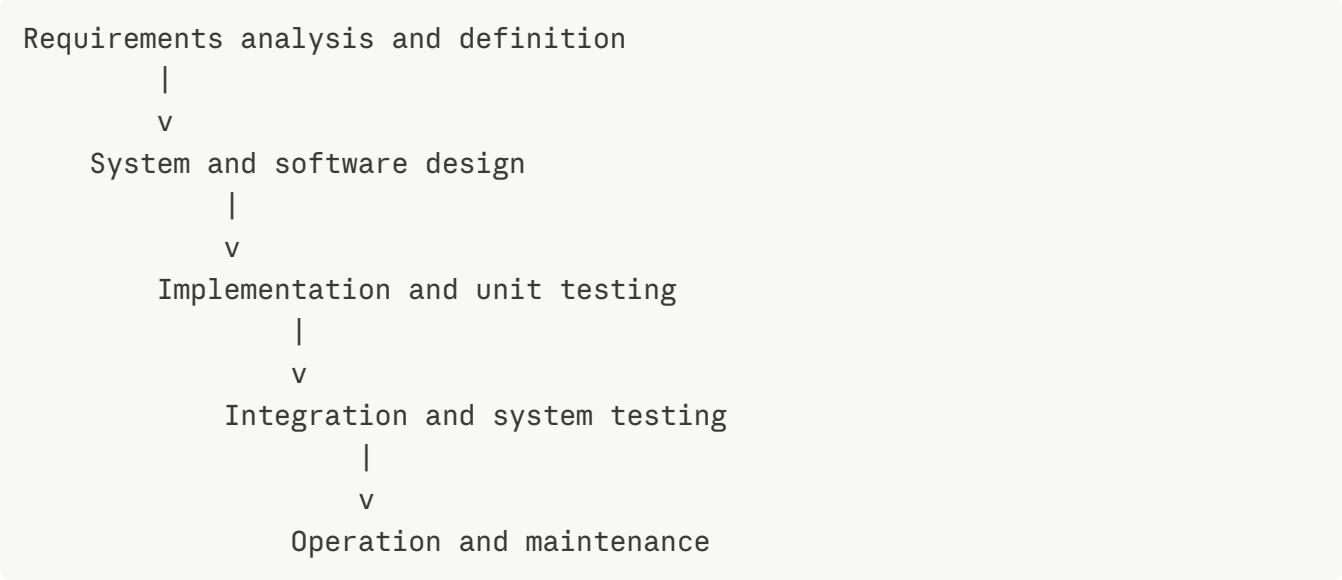
1. The system shall allow a user to register and log in. (From the ShopFast example: basic user registration and login.)
2. The system shall allow a user to search and filter the product catalog. (From the ShopFast example: product search and filter functionality.)

Non-functional:

1. **Product requirement:** The system shall process each user search and page request within 2 seconds. (Speed: user/event response time.)
2. **External requirement:** The system shall implement customer payment-data privacy provisions as required by law. (External requirements arise from legislative requirements.)

Q2.a (8 marks) With diagram, explain in detail about the waterfall process model for software development

- The waterfall model is an example of a plan-driven process. Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- It has separate and distinct phases of specification and development.



Waterfall model phases (there are separate identified phases):

1. Requirements analysis and definition
2. System and software design
3. Implementation and unit testing
4. Integration and system testing
5. Operation and maintenance

Drawback / problems:

- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.
- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
- Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process. Few business systems have stable requirements.

Where it is used: The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites. In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Q2.b (7 marks) What are the common challenges faced during requirements elicitation, and how can a software engineer mitigate them?

Requirements elicitation involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints. It may involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders.

Challenges (problems of requirements elicitation):

1. Stakeholders don't know what they really want.
2. Stakeholders express requirements in their own terms.
3. Different stakeholders may have conflicting requirements.
4. Organisational and political factors may influence the system requirements.
5. The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.
6. *Domain requirements:* Requirements are expressed in the language of the application domain, which is often not understood by software engineers (Understandability). Domain specialists understand the area so well that they do not think of making the domain requirements explicit (Implicitness).

How a software engineer can mitigate them:

- **Interviewing:** Formal or informal interviews with stakeholders are part of most RE processes. Use a mix of closed interviews (pre-determined list of questions) and open interviews (various issues are explored). Be open-minded, avoid pre-conceived ideas about the requirements and be willing to listen to stakeholders. Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system.
- **Scenarios:** Real-life examples of how a system can be used. They include the starting situation, the normal flow of events, what can go wrong, other concurrent activities and the state when the scenario finishes.
- **Use cases:** A scenario based technique in the UML which identifies the actors in an interaction and describes the interaction itself.
- **Ethnography:** A social scientist spends a considerable time observing and analysing how people actually work. People do not have to explain or articulate their work. This helps where interviews fail on domain knowledge that people find hard to articulate.
- **Prototyping:** Helps with requirements elicitation and validation. Focused ethnography combines ethnography with prototyping.
- **Requirements prioritisation and negotiation:** Prioritising requirements and resolving requirements conflicts.

- **Requirements classification and organisation:** Groups related requirements and organises them into coherent clusters.
- **Requirements management:** Keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. Establish a formal process for making change proposals.

Q3.a (6 marks) Describe and elaborate the four distinct phases of the Rational Unified Process (RUP), with a neat diagram

- The Rational Unified Process is a modern generic process derived from the work on the UML and associated process. It brings together aspects of the 3 generic process models.
- It is normally described from 3 perspectives: a dynamic perspective that shows phases over time; a static perspective that shows process activities; a practice perspective that suggests good practice.

```
Inception --> Elaboration --> Construction --> Transition
(Establish     (Problem        (System design,   (Deploy the
 business       domain and      programming       system)
 case)          architecture)   and testing)
  ^   In-phase iteration: each phase is iterative            |
  |_____ Cross-phase iteration: whole set of phases ________|
```

RUP phases:

- **Inception:** Establish the business case for the system.
- **Elaboration:** Develop an understanding of the problem domain and the system architecture.
- **Construction:** System design, programming and testing.
- **Transition:** Deploy the system in its operating environment.

RUP iteration:

- **In-phase iteration:** Each phase is iterative with results developed incrementally.
- **Cross-phase iteration:** As shown by the loop in the RUP model, the whole set of phases may be enacted incrementally.

Static workflows (for extra points): Business modelling, Requirements, Analysis and design, Implementation, Testing, Deployment, Configuration and change management, Project management, Environment. The RUP is organized into phases (inception, elaboration, construction and transition) but separates activities (requirements, analysis and design, etc.) from these phases.

Q3.b (5 marks) Create a simple Use Case Diagram for a "Library Management System" involving actors like Member and Librarian, with at least three key use cases

Use cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself. Each use case represents a discrete task that involves external interaction with a system. Actors in a use case may be people or other systems.

```
+---------- Library Management System ----------+
|   (Search book)   (Issue book)   (Return book) |
|   (Manage members)  (Add or remove books)      |
+------------------------------------------------+
Member    ---- Search book, Issue book, Return book
Librarian ---- Search book, Issue book, Return book,
               Manage members, Add or remove books
```

Actors: Member and Librarian (people).

Key use cases: Search book, Issue book, Return book (Member and Librarian); Manage members, Add or remove books (Librarian).

Q3.c (4 marks) List and briefly explain requirement validation process

- Requirements validation is concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important. Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking:

1. **Validity.** Does the system provide the functions which best support the customer's needs?
2. **Consistency.** Are there any requirements conflicts?
3. **Completeness.** Are all functions required by the customer included?
4. **Realism.** Can the requirements be implemented given available budget and technology?
5. **Verifiability.** Can the requirements be checked?

Requirements validation techniques:

1. **Requirements reviews:** Systematic manual analysis of the requirements.
2. **Prototyping:** Using an executable model of the system to check requirements.
3. **Test-case generation:** Developing tests for requirements to check testability.

Requirements reviews: Regular reviews should be held while the requirements definition is being formulated. Both client and contractor staff should be involved in reviews. Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks: Verifiability (Is the requirement realistically testable?), Comprehensibility (Is the requirement properly understood?), Traceability (Is the origin of the requirement clearly stated?), Adaptability (Can the requirement be changed without a large impact on other requirements?).

What isn't verbatim from your slides: the e-commerce examples in Q1.b and the Library Management System diagram in Q3.b are my own, since the slides only give MHC-PMS examples; the mitigation pairing in Q2.b maps the slides' elicitation techniques onto the listed problems.

Part D – Extra important topics from the slides (not yet asked in the papers)

These were picked from the slides themselves, not from the question papers. They are good candidates for 'short note' or 6–8 mark questions.

#	Topic	Slides
D1	Software products: generic vs customized, product specification	U1.1: 5–6
D2	Essential attributes of good software; FAQs on SE	U1.1: 7–9
D3	Software process, process activities	U1.2: 3–4, 22–33; U1.1: 12
D4	Reuse-oriented software engineering	U1.2: 19–21
D5	Coping with change, prototyping, throw-away prototypes	U1.2: 37–43
D6	Incremental delivery	U1.2: 44–48
D7	Scrum	U1.3: 38–43
D8	XP testing, pair programming, refactoring	U1.3: 23–25, 28–36
D9	Agile and maintenance; scaling agile	U1.3: 9, 44–49
D10	Application types and general issues	U1.1: 13–17
D11	Requirements imprecision, completeness, consistency; guidelines	U2.1: 12–13, 37

#	Topic	Slides
D12	Changing requirements and requirements management planning	U2.1: 76–79
D13	Software design and implementation, validation and testing stages	U1.2: 25–31

D1. Generic vs customized products (U1.1 slides 5–6)

- **Generic products:** Stand-alone systems that are marketed and sold to any customer who wishes to buy them. Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists. The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.
- **Customized products:** Software that is commissioned by a specific customer to meet their own needs. Examples – embedded control systems, air traffic control software, traffic monitoring systems. The specification is owned by the customer for the software and they make decisions on software changes that are required.

D2. Attributes of good software (U1.1 slides 7–9)

- Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
- **Maintainability:** Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
- **Dependability and security:** includes reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
- **Efficiency:** Software should not make wasteful use of system resources such as memory and processor cycles. Includes responsiveness, processing time, memory utilisation, etc.
- **Acceptability:** Software must be acceptable to the type of users for which it is designed: understandable, usable and compatible with other systems that they use.
- **Key challenges (slide 8):** Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
- **Costs (slide 8):** Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.

- **SE vs computer science:** Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
- **SE vs system engineering:** System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

D3. Software process and process activities (U1.2 slides 3–4, 22–33)

- A software process is a structured set of activities required to develop a software system. All involve: Specification – defining what the system should do; Design and implementation – defining the organization of the system and implementing the system; Validation – checking that it does what the customer wants; Evolution – changing the system in response to changing customer needs.
- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.
- Process descriptions may also include: Products (outcomes of a process activity); Roles (responsibilities of the people involved); Pre- and post-conditions.
- **Design activities (slide 27):** Architectural design, Interface design, Component design, Database design.
- **Validation (slides 28–30):** Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer. Testing stages: Development or component testing; System testing; Acceptance testing.
- **Evolution (slide 32):** Software is inherently flexible and can change. As requirements change through changing business circumstances, the software that supports the business must also evolve and change.

D4. Reuse-oriented software engineering (U1.2 slides 19–21)

- Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems.
- **Process stages:** Component analysis; Requirements modification; System design with reuse; Development and integration.
- It prioritizes the use of existing software components over building everything from scratch. Building with proven, pre-tested components can drastically reduce development time, cost, and effort while increasing overall system reliability and quality.

- **Types of component:** Web services developed according to service standards and available for remote invocation; collections of objects developed as a package to be integrated with a component framework such as .NET or J2EE; stand-alone software systems (COTS) configured for use in a particular environment.

D5. Coping with change, prototyping (U1.2 slides 37–43)

- Change is inevitable in all large software projects. Business changes lead to new and changed system requirements; new technologies open up new possibilities; changing platforms require application changes.
- Change leads to rework, so the costs of change include both rework (e.g. re-analysing requirements) as well as the costs of implementing new functionality.
- **Change avoidance:** the software process includes activities that can anticipate possible changes before significant rework is required, for example, a prototype system may be developed to show some key features of the system to customers.
- **Change tolerance:** the process is designed so that changes can be accommodated at relatively low cost. This normally involves some form of incremental development.
- **Prototype development (slide 42):** may be based on rapid prototyping languages or tools; may involve leaving out functionality; should focus on areas of the product that are not well-understood; error checking and recovery may not be included; focus on functional rather than non-functional requirements.
- **Throw-away prototypes (slide 43):** Prototypes should be discarded after development as they are not a good basis for a production system: it may be impossible to tune the system to meet non-functional requirements; prototypes are normally undocumented; the prototype structure is usually degraded through rapid change; the prototype probably will not meet normal organisational quality standards.

D6. Incremental delivery (U1.2 slides 44–48)

- Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality. User requirements are prioritised and the highest priority requirements are included in early increments. Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.
- **Advantages (slide 47):** Customer value can be delivered with each increment so system functionality is available earlier. Early increments act as a prototype to help elicit requirements for later increments. Lower risk of overall project failure. The highest priority system services tend to receive the most testing.

- **Problems (slide 48):** Most systems require a set of basic facilities that are used by different parts of the system; as requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities. This conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.

D7. Scrum (U1.3 slides 38–43)

- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.
- **Three phases:** the initial outline planning phase, where you establish the general objectives for the project and design the software architecture; a series of sprint cycles, where each cycle develops an increment of the system; the project closure phase, which wraps up the project, completes required documentation and assesses the lessons learned.
- **Sprint cycle (slides 40–41):** Sprints are fixed length, normally 2–4 weeks. The starting point for planning is the product backlog, which is the list of work to be done on the project. The selection phase involves all of the project team who work with the customer to select the features to be developed during the sprint. The team is isolated from the customer and the organization, with all communications channelled through the Scrum master, whose role is to protect the development team from external distractions. At the end of the sprint, the work done is reviewed and presented to stakeholders.
- **Teamwork (slide 42):** The Scrum master is a facilitator who arranges daily meetings, tracks the backlog of work, records decisions, measures progress against the backlog and communicates with customers and management outside of the team. Short daily meetings share progress, problems and plans for the next day.
- **Benefits (slide 43):** The product is broken down into manageable chunks. Unstable requirements do not hold up progress. The whole team have visibility of everything, so communication is improved. Customers see on-time delivery of increments and gain feedback. Trust between customers and developers is established.

D8. XP testing, pair programming, refactoring (U1.3 slides 23–25, 28–36)

- **Testing in XP (slides 28–29):** XP has developed an approach where the program is tested after every change has been made. Features: test-first development; incremental test development from scenarios; user involvement in test development and validation; automated test harnesses run all component tests each time a new release is built. Writing tests before code clarifies the requirements to be implemented. Usually relies on a testing framework such as Junit.

- **Test difficulties (slide 33):** Programmers prefer programming to testing and sometimes take short cuts; some tests are very difficult to write incrementally (e.g. display logic in a complex user interface); it is difficult to judge the completeness of a set of tests.
- **Pair programming (slides 34–36):** Programmers sit together at the same workstation to develop the software. Advantages: it supports collective ownership and responsibility for the system; it acts as an informal review process because each line of code is looked at by at least two people; it helps support refactoring; the sharing of knowledge reduces the overall risks to a project when team members leave.
- **Refactoring (slides 24–25):** The programming team look for possible software improvements and make these improvements even where there is no immediate need for them. This improves the understandability of the software and so reduces the need for documentation. Examples: re-organization of a class hierarchy to remove duplicate code; tidying up and renaming attributes and methods; replacing inline code with calls to library methods. Some changes require architecture refactoring and this is much more expensive.

D9. Agile and maintenance; scaling agile (U1.3 slides 9, 44–49)

- **Maintenance (slide 9):** Most organizations spend more on maintaining existing software than on new development. Two key issues: Are systems developed using an agile approach maintainable, given the emphasis on minimizing formal documentation? Can agile methods be used effectively for evolving a system in response to customer change requests? Problems may arise if the original development team cannot be maintained.
- **Scaling:** 'Scaling up' is concerned with using agile methods for developing large software systems that cannot be developed by a small team. 'Scaling out' is concerned with how agile methods can be introduced across a large organization with many years of software development experience. When scaling, it is essential to maintain agile fundamentals: flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.
- **Large systems (slides 45–46):** collections of separate, communicating systems developed by separate teams in different places; 'brownfield systems' that interact with existing systems; constrained by external rules and regulations; long procurement and development time; diverse set of stakeholders.
- **Scaling up (slide 48):** you need to do more up-front design and system documentation; cross-team communication mechanisms; continuous integration is practically impossible, but frequent system builds and regular releases are essential.

- **Scaling out (slide 49):** managers without agile experience may be reluctant; quality procedures and standards may be incompatible with agile methods; a wide range of skills and abilities; cultural resistance.

D10. Application types and general issues (U1.1 slides 13–17)

- **General issues:** Heterogeneity (systems operate as distributed systems across networks that include different types of computer and mobile devices); Business and social change; Security and trust.
- **Application types:** Stand-alone applications; Interactive transaction-based applications (e.g. e-commerce); Embedded control systems; Batch processing systems; Entertainment systems; Systems for modeling and simulation; Data collection systems; Systems of systems. Definitions are on slides 15–17.

D11. Imprecision, completeness, consistency, writing guidelines (U2.1 slides 12–13, 37)

- **Imprecision:** Problems arise when requirements are not precisely stated. Ambiguous requirements may be interpreted in different ways by developers and users (e.g. the term ‘search’ in the MHC-PMS example).
- **Complete:** include descriptions of all facilities required. **Consistent:** no conflicts or contradictions. In practice, it is impossible to produce a complete and consistent requirements document.
- **Guidelines (slide 37):** Invent a standard format and use it for all requirements. Use language in a consistent way: shall for mandatory requirements, should for desirable requirements. Use text highlighting to identify key parts. Avoid the use of computer jargon. Include an explanation (rationale) of why a requirement is necessary.

D12. Changing requirements and requirements management planning (U2.1 slides 76–79)

- The business and technical environment of the system always changes after installation: new hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change, and new legislation and regulations may be introduced.
- The people who pay for a system and the users of that system are rarely the same people. Large systems usually have a diverse user community with conflicting priorities; the final system requirements are inevitably a compromise.
- **Requirements management planning (slide 79):** Requirements identification (each requirement uniquely identified); A change management process; Traceability policies; Tool support.

D13. Design and implementation, testing stages (U1.2 slides 25–31)

- Software design and implementation is the process of converting the system specification into an executable system. Design a software structure that realises the specification; implementation translates this structure into an executable program. The activities of design and implementation are closely related and may be inter-leaved.
- **Design activities:** Architectural design, Interface design, Component design, Database design.
- **Testing stages:** Development or component testing (individual components tested independently); System testing (testing of the system as a whole, emergent properties are particularly important); Acceptance testing (testing with customer data to check that the system meets the customer's needs).

How to use this file

1. **Part A** tells you what matters and where it is in the slides.
2. **Part B** has the answers for the SEE questions (Units 1 and 2).
3. **Part C** has the CIE-1 answers.
4. **Part D** covers important slide topics that have not been asked yet.